

Relational Design Theory

Functional Dependencies · BCNF · Multivalued Dependencies · 4NF

What is Relational Design Theory?

- When designing a database, many schema designs are possible — some much better than others
- Goal: Find a principled way to choose a good schema automatically
- Two main tools:
 - → Functional Dependencies (FDs) $\Rightarrow$ Boyce-Codd Normal Form (BCNF)
 - → Multivalued Dependencies (MVDs) $\Rightarrow$ Fourth Normal Form (4NF)
- Approach: Design by Decomposition — start with 'mega' relations, break down using dependency rules
- Guarantees: No anomalies, no lost information, all facts stored exactly once

According to Widom (2012), The designer specifies properties; the system does the decomposition.

Section 1

Motivation & Design Anomalies

Why bad schemas cause real problems

Motivation: A Bad Schema Design

- College application data for student Ann (SSN 123):
 - Attended PAHS and GHS (both in Palo Alto)
 - Plays tennis and trumpet
 - Applies to Stanford, Berkeley, MIT
- Naïve design: Apply(SSN, sName, cName, HSname, HScity, hobby)
- → Needs 12 tuples: 1 student $\times$ 2 high schools $\times$ 2 hobbies $\times$ 3 colleges
- Each fact repeated many times — this is the root problem!
- SSN-name pair stored 12 times; 'Ann attends PAHS' stored 6 times

 *Redundancy = every piece of information captured many more times than necessary.*

Three Design Anomalies

- ① REDUNDANCY
 - Each fact stored multiple times (e.g., Ann's name appears in all 12 tuples)
 - Wastes storage; inconsistency risk
- ② UPDATE ANOMALY
 - Changing one fact requires updating ALL copies
 - If only 3 of 12 copies updated → database becomes inconsistent
- ③ DELETION ANOMALY
 - Deleting one piece of data may inadvertently remove other data
 - Remove 'surfing' hobby tuples → student may disappear entirely from DB

 These anomalies are direct symptoms of a poor relational schema design.

A Better Design — Decomposed Relations

- Split the mega-relation into smaller, purpose-built relations:
 - Student(SSN, sName)
 - Apply(SSN, cName)
 - HighSchool(SSN, HSname, HScity)
 - Hobbies(SSN, hobby)
- Benefits:
 - ✓ No redundancy — each fact stored exactly once
 - ✓ No update or deletion anomalies
 - ✓ Original mega-relation can be reconstructed by joining
- The decomposition preserves all information while eliminating anomalies

 *The best design depends on what the data represents in the real world.*

Design by Decomposition — The Big Idea

- Step 1: Start with 'mega' relations containing all attributes
 - Step 2: Specify formal properties of the data (FDs or MVDs)
 - Step 3: System automatically decomposes based on those properties
 - Step 4: Final relations satisfy a Normal Form
-
- Normal Form guarantees:
 - No anomalies (redundancy, update, deletion)
 - Lossless decomposition (original relation fully reconstructible)
-
- Functional Dependencies → Boyce-Codd Normal Form (BCNF)
 - Functional + Multivalued Dependencies → Fourth Normal Form (4NF)

Section 2

Functional Dependencies

The formal basis for detecting schema problems

Functional Dependency — Definition

$A \rightarrow B$ means: any two tuples with same A values must have the same B values

- Formally: if $t[\bar{A}] = u[\bar{A}]$ then $t[B] = u[B]$ (for all tuples t, u in relation R)
- FDs are based on knowledge of the real world — not derived from data alone
- All instances of the relation must adhere to every specified FD
- Multiple attributes allowed on each side: $A_1, A_2, \dots \rightarrow B_1, B_2, \dots$
- FD generalises the concept of keys
- Examples from Student(SSN, sName, address, HScore, HSname, HScity, GPA, priority):
 - $SSN \rightarrow sName$ (each SSN belongs to exactly one student)
 - $SSN \rightarrow address$ (one address per student)
 - $HScore \rightarrow HSname, HScity$ (each school code identifies one school)
 - $GPA \rightarrow priority$ (priority is a function of GPA)

Types of Functional Dependencies

- TRIVIAL FD: $A \rightarrow B$ where $B \subseteq A$
 - $\rightarrow$ RHS is already contained within LHS
 - $\rightarrow$ Always satisfied; provides no new information
 - $\rightarrow$ Example: $\{\text{SSN}, \text{sName}\} \rightarrow \text{sName}$
- NONTRIVIAL FD: $A \rightarrow B$ where $B \not\subseteq A$ (at least some B not in A)
 - $\rightarrow$ Provides real constraint on the data
- COMPLETELY NONTRIVIAL FD: $A \rightarrow B$ where $A \cap B = \emptyset$
 - $\rightarrow$ No overlap between LHS and RHS
 - $\rightarrow$ These are the dependencies we really care about for design!
- Goal: Find a minimal set of completely nontrivial FDs from which all others follow

Rules for Functional Dependencies

- SPLITTING RULE: $A \rightarrow B_1, B_2, \dots B_n \Rightarrow A \rightarrow B_1, A \rightarrow B_2, \dots A \rightarrow B_n$
 - (split the RHS — each attribute can be separated)
 - ⚠ Splitting does NOT work on LHS!
- COMBINING RULE: $A \rightarrow B$ and $A \rightarrow C \Rightarrow A \rightarrow B, C$
 - (merge multiple FDs with the same LHS)
- TRIVIAL-DEPENDENCY RULE:
 - $A \rightarrow B \Rightarrow A \rightarrow A, B$ (add LHS attributes to RHS)
 - $A \rightarrow A, B \Rightarrow A \rightarrow B$ (remove LHS attributes from RHS)
- TRANSITIVE RULE: $A \rightarrow B$ and $B \rightarrow C \Rightarrow A \rightarrow C$
 - Example: $SSN \rightarrow GPA$ and $GPA \rightarrow priority \Rightarrow SSN \rightarrow priority$

💡 These rules are all formally provable. Armstrong's Axioms provide a complete set.

Closure of Attributes

The closure of attribute set $\bar{A}$ (written $\bar{A}^+$) = all attributes B such that $\bar{A} \rightarrow B$

- Given: a relation R , its FDs, and a set of attributes $\bar{A}$
- Find ALL attributes that are functionally determined by $\bar{A}$
- ALGORITHM (Closure):
 1. Start with the set $\bar{A}$
 2. Repeat until no change:
 - If $X \rightarrow Y$ is an FD and X is in the current set $\rightarrow$ add Y to the set
 3. The final set is $\bar{A}^+$
- Example — Student(SSN, sName, address, HScore, HSname, HScity, GPA, priority)
- FDs: SSN $\rightarrow$ sName, address, GPA | GPA $\rightarrow$ priority | HScore $\rightarrow$ HSname, HScity
- Closure of {SSN, HScore} $^+$:
 - Start: {SSN, HScore} $\rightarrow$ add sName, address, GPA $\rightarrow$ add priority $\rightarrow$ add HSname, HScity

Closure and Keys

- Key Property: If $\bar{A}^+ = \text{all attributes of } R$, then $\bar{A}$ is a key for R
- Using closure to test if $\bar{A}$ is a key:
 - Compute $\bar{A}^+$ using the FDs $\rightarrow$ if $\bar{A}^+ = \text{all attributes}$ $\rightarrow$ $\bar{A}$ is a key
- Finding ALL keys:
 - Enumerate every subset of attributes; compute its closure
 - If closure = all attributes $\rightarrow$ that subset is a key
 - Optimisation: once $\bar{A}$ is found to be a key, every superset of $\bar{A}$ is also a key
 - Work in order of increasing subset size for efficiency
- If two tuples agree on $\bar{A}$ and $\bar{A}$ functionally determines all attributes
 - $\rightarrow$ the tuples must be identical $\rightarrow$ $\bar{A}$ uniquely identifies every tuple
 - $\rightarrow$ Therefore $\bar{A}$ is a key

Section 3

Boyce-Codd Normal Form (BCNF)

Eliminating redundancy using functional dependencies

Boyce-Codd Normal Form — Definition

Relation R is in BCNF if: for every FD $A \rightarrow B$, A is a key of R

- Equivalently: the only non-trivial FDs allowed are those where LHS is a key
- Why does this eliminate anomalies?
- If $A \rightarrow B$ but A is NOT a key, two tuples with same A can exist
- Those tuples share the same B value $\rightarrow$ information stored twice $\rightarrow$ redundancy!
- BCNF prevents this by requiring A to uniquely identify each tuple
- BCNF Violation Example:
- Apply(SSN, sName, cName) with FD $SSN \rightarrow sName$
- SSN is NOT the key (key = {SSN, cName})
- $\rightarrow$ Student 123 named Ann stored once per college applied to $\rightarrow$ VIOLATION
- Fix (BCNF decomposition):
- Student(SSN, sName) $\leftarrow$ SSN is key ✓

BCNF Decomposition Algorithm

- INPUT: Relation R with its set of functional dependencies
- OUTPUT: Lossless decomposition into BCNF relations
- ALGORITHM:
 1. Compute key(s) for R using FD closure
 2. Repeat until all relations are in BCNF:
 - a. Pick any relation R' with a violating FD: $A \rightarrow B$ (A is not a key)
 - b. Decompose R' into:
 - $R_1 = (A \cup B) \leftarrow$ the attributes in the FD
 - $R_2 = (R' - B) \cup A \leftarrow$ LHS + remaining attributes
 - c. Compute FDs and keys for R_1 and R_2
 3. Return the set of decomposed relations
- Lossless join property: $R = R_1 \bowtie R_2$ (always guaranteed by BCNF decomposition)
- Note: algorithm is non-deterministic — different FD choices yield different valid BCNFs

BCNF Decomposition — Worked Example

- Start: Student(SSN, sName, address, HScore, HSname, HScity, GPA, priority)
- FDs: $SSN \rightarrow sName, address, GPA$ | $GPA \rightarrow priority$ | $HScode \rightarrow HSname, HScity$
- Key: {SSN, HScore} (all FDs violate BCNF)
- Step 1: Use $HScode \rightarrow HSname, HScity$
- $S1(HScore, HSname, HScity) \leftarrow HScode$ is key ✓ [BCNF]
- $S2(SSN, sName, address, HScore, GPA, priority)$ [still violates]
- Step 2: Use $GPA \rightarrow priority$ on S2
- $S3(GPA, priority) \leftarrow GPA$ is key ✓ [BCNF]
- $S4(SSN, sName, address, HScore, GPA)$ [still violates]
- Step 3: Use $SSN \rightarrow sName, address, GPA$ on S4
- $S5(SSN, sName, address, GPA) \leftarrow SSN$ is key ✓ [BCNF]
- $S6(SSN, HScore) \leftarrow \{SSN, HScore\}$ is key ✓ [BCNF]
- Final schema: S1, S3, S5, S6 — each relation in BCNF, no anomalies!

Section 4

Multivalued Dependencies & 4NF

Handling independent repeating facts

Why BCNF is Sometimes Not Enough

- Consider: Apply(SSN, cName, hobby)
- Assumptions: students apply to multiple colleges; students have multiple hobbies
- College applications and hobbies are INDEPENDENT of each other

- FDs? → None (SSN doesn't determine cName or hobby uniquely)
- Key? → All three attributes together: {SSN, cName, hobby}
- In BCNF? → YES (no non-trivial FDs to violate BCNF)
- Good design? → NO!

- Problem: 5 colleges $\times$ 6 hobbies = 30 tuples for one student
- We'd prefer $m + n$ tuples ($5 + 6 = 11$), not $m \times n = 30$
- This multiplicative redundancy is NOT captured by FDs → Need MVDs

💡 *BCNF handles FD-based redundancy; 4NF handles independence-based redundancy.*

Multivalued Dependency — Definition

$A \twoheadrightarrow B$ means: for a given A, B values are independent of all other attributes

- Formal definition: if tuples t and u agree on A , then there exists a tuple v such that:
 - $v[A] = t[A] = u[A]$
 - $v[B] = t[B]$ (takes B from t)
 - $v[\text{rest}] = u[\text{rest}]$ (takes remaining attributes from u)
- Intuition: for any value of A , ALL combinations of B and (other attributes) appear
- Example: Apply(SSN, cName, hobby) with $\text{SSN} \twoheadrightarrow \text{cName}$
 - If (123, Stanford, tennis) and (123, Berkeley, trumpet) exist in R
 - $\rightarrow$ (123, Stanford, trumpet) must also exist in R
 - $\rightarrow$ (123, Berkeley, tennis) must also exist in R
- Note: $\text{SSN} \twoheadrightarrow \text{cName}$ also implies $\text{SSN} \twoheadrightarrow \text{hobby}$ (complementation rule)

Rules for Multivalued Dependencies

- RULE 1 — FD-is-an-MVD:
 - If $A \rightarrow B$ then $A \twoheadrightarrow B$
 - Every FD is also an MVD (FDs are a special case of MVDs)
 - This means: 4NF is strictly stronger than BCNF
- RULE 2 — Intersection Rule:
 - If $A \twoheadrightarrow B$ and $A \twoheadrightarrow C$ then $A \twoheadrightarrow (B \cap C)$
- RULE 3 — Transitive Rule (modified):
 - If $A \twoheadrightarrow B$ and $B \twoheadrightarrow C$ then $A \twoheadrightarrow (C - B)$
- TRIVIAL MVD: $A \twoheadrightarrow B$ is trivial if $B \subseteq A$ or $A \cup B = \text{all attributes}$
- NONTRIVIAL MVD: otherwise
-  Splitting rule applies to FDs but NOT always to MVDs
 - (FDs are a subset of MVDs — rules for MVDs are not automatically rules for FDs)

Fourth Normal Form — Definition

Relation R is in 4NF if: for every nontrivial MVD $A \twoheadrightarrow B$, A is a key

- 4NF implies BCNF (since every FD is also an MVD, BCNF conditions are also 4NF conditions)
- 4NF is STRONGER than BCNF — fewer relations satisfy 4NF
- Why does 4NF eliminate the multiplicative redundancy?
 - If $A \twoheadrightarrow B$ but A is not a key $\rightarrow$ multiple tuples with same A exist
 - $\rightarrow$ all combinations of B and other attributes must appear $\rightarrow$ exponential blowup!
 - If A IS a key $\rightarrow$ only one tuple per A value $\rightarrow$ no blowup
- Fix for Apply(SSN, cName, hobby):
 - Violating MVD: $SSN \twoheadrightarrow cName$ (SSN is not a key)
 - Decompose into:
 - Apply(SSN, cName) $\leftarrow$ {SSN, cName} is key ✓ [4NF]
 - Hobbies(SSN, hobby) $\leftarrow$ {SSN, hobby} is key ✓ [4NF]

4NF Decomposition Algorithm

- INPUT: Relation R with FDs and MVDs
- OUTPUT: Lossless decomposition into 4NF relations
- ALGORITHM:
 1. Compute keys for R using FDs (FDs $\rightarrow$ keys, not MVDs)
 2. Repeat until all relations are in 4NF:
 - a. Pick any R' with a nontrivial violating MVD: $A \twoheadrightarrow B$ (A is not a key)
 - b. Decompose R' into:
 - $R_1 = A \cup B \quad \leftarrow$ attributes in the MVD
 - $R_2 = R' - B \cup A \quad \leftarrow$ LHS + remaining attributes
 - c. Compute FDs and MVDs for R_1 and R_2
 - d. Compute keys for R_1 and R_2
- Same structure as BCNF algorithm, but uses MVD violations instead of FD violations
- Note: Finding MVDs for sub-relations can be complex — usually intuitive in practice

4NF Decomposition — Worked Example

- Start: Apply(SSN, cName, date, major, hobby)
- FD: {SSN, cName} $\rightarrow$ date (apply once per college, on one date)
- MVD: {SSN, cName, date} $\twoheadrightarrow$ major (majors and hobbies are independent)
- Key: All 5 attributes $\rightarrow$ both FD and MVD violate 4NF

- Step 1: Use MVD {SSN,cName,date} $\twoheadrightarrow$ major to decompose:
 - A1(SSN, cName, date, major) A2(SSN, cName, date, hobby)

- Step 2: FD {SSN,cName} $\rightarrow$ date still violates in A1 and A2 — decompose both:
 - From A1: A3(SSN, cName, date) A4(SSN, cName, major)
 - From A2: A3(SSN, cName, date) [same!] A5(SSN, cName, hobby)

- Final schema:
 - A3(SSN, cName, date) A4(SSN, cName, major) A5(SSN, cName, hobby)
 - All three are in 4NF — independent facts stored separately!

Section 5

Shortcomings of BCNF/4NF

When good theory meets real-world trade-offs

Shortcomings of BCNF and 4NF

- Problem 1 — LOST DEPENDENCIES
 - Decomposition can split apart FDs across different relations
 - Enforcing an FD then requires joining multiple relations → expensive queries
 - Example: Apply(SSN,cName,date,major) with $\text{date} \rightarrow \text{cName}$
 - BCNF decomposes into $A1(\text{cName}, \text{date}) + A2(\text{SSN}, \text{cName}, \text{major})$
 - But now checking $\{\text{SSN}, \text{cName}\} \rightarrow \text{major}$ requires joining A1 and A2
- Problem 2 — QUERY WORKLOAD
 - If most queries need all attributes anyway, joins become expensive
 - Denormalised relations (intentionally NOT in BCNF/4NF) may perform better
 - Consider: Student(SSN,sName,SAT,WAEC) — if all queries use all columns,
 - decomposing into 3 tables forces every query to do 2 joins
- Problem 3 — OVER-DECOMPOSITION

 Third Normal Form (3NF) is a weaker but dependency-preserving alternative to BCNF.

But unnecessarily splitting data into 3 tables when 1 would do is poor design

Hierarchy of Normal Forms

- All relational schemas
 - $\sqsubset$ 1NF: atomic values in every cell (every table satisfies this)
 - $\sqsubset$ 2NF: no partial dependencies on a composite key
 - $\sqsubset$ 3NF: no transitive dependencies — slightly weaker than BCNF
 - $\sqsubset$ BCNF: every non-trivial FD has a key on LHS
 - $\sqsubset$ 4NF: every non-trivial MVD has a key on LHS
- $4NF \subset BCNF \subset 3NF \subset 2NF \subset 1NF$
- In this course, focus is on BCNF (FDs only) and 4NF (FDs + MVDs)
- 3NF is used when dependency preservation is essential
- 1NF/2NF are historical stepping stones — rarely discussed in modern design

Section 6

Summary

Relational Design Theory at a Glance

BCNF vs 4NF — At a Glance

BCNF (Boyce-Codd Normal Form)

- Based on Functional Dependencies
- FD: $A \rightarrow B$ means same $A \rightarrow$ same B
- Rule: every non-trivial FD must have key on LHS
- Eliminates: FD-based redundancy
- Algorithm: decompose on violating FD
- Guarantees: lossless join, no update/deletion anomalies
- Weakness: may lose some FD constraints

4NF (Fourth Normal Form)

- Based on Functional + Multivalued Dependencies
- MVD: $A \twoheadrightarrow B$ means A causes all $B \times (\text{rest})$ combinations
- Rule: every non-trivial MVD must have key on LHS
- Eliminates: both FD-based and MVD-based redundancy
- Algorithm: decompose on violating MVD (or FD)
- Stronger: $4NF \subset BCNF$ — every 4NF relation is in BCNF
- Weakness: even more query overhead from additional joins

The Complete Design by Decomposition Process

- 1. Model the domain — identify all relevant attributes
- 2. Create mega-relation(s) containing all attributes
- 3. Specify constraints as FDs (and MVDs if needed):
 - → Interview domain experts; reason about real-world rules
 - → Find a minimal, completely nontrivial set
- 4. Compute keys using closure algorithm
- 5. Test each FD/MVD for BCNF/4NF violations
- 6. Decompose iteratively using the appropriate algorithm
- 7. Verify:
 - ✓ Lossless join property holds
 - ✓ All decomposed relations are in the target normal form
 - ✓ Query workload is manageable (consider denormalisation if needed)
- 8. Repeat from step 5 until all relations satisfy the normal form

Key Takeaways

Functional Dependencies formalise 'knowing one attribute tells you another'

Closure algorithm efficiently finds all implied FDs and identifies keys

BCNF eliminates all FD-based redundancy by requiring FD left-sides to be keys

Multivalued Dependencies capture independent repeating facts

4NF extends BCNF by also eliminating MVD-based redundancy

Design by decomposition is automatic but trade-offs exist (dependency loss, query overhead)